

NLP

Text Preprocessing and Text Representations

Contents

Introduction to Text Preprocessing

Techniques of Text preprocessing:

- ✓ Tokenization
- ✓ Stop word Removal
- ✓ Stemming and lemmatization
- ✓ Parts of speech tagging

Text Representation in NLP

- ✓ One Hot Encoding
- ✓ Bag of words
- ✓ TF-IDF (Term Frequency – Inverse Document Frequency)

What is Text preprocessing

Text preprocessing is an essential step in Natural Language Processing (NLP) that involves cleaning and transforming **unstructured text data** to prepare it for analysis.



Free-form and human-generated
Varies in length, style, grammar, and vocabulary
Often noisy (typos, emojis, abbreviations)

Various techniques of text preprocessing are:

1. Tokenization
2. Stop word Removal
3. Stemming and lemmatization
4. Parts of speech tagging

Text Preprocessing in NLP

Tokenization

Tokenization is a fundamental process in Natural Language Processing (NLP) that involves breaking down a stream of text into smaller units called tokens.

Sentence Tokenization

“I like mangoes. They are sweet.”

Tokens : ["I like mangoes.", "They are sweet."]

Word Tokenization

“I like mangoes”

Tokens: ["I", "like", "mangoes"]

Subword Tokenization

“I like mangoes”

Tokens: ["I", "like", "man", "##goes"]

Character Tokenization

“I like mangoes”

Tokens:

["I", ",", " ", "l", "i", "k", "e", " ", "m", "a", "n", "g", "o", "e", "s"]

Stop Word Removal

- These are the most common words in any language (like articles, prepositions, pronouns, conjunctions, etc
- They does not add much information to the text. Removing them improves computational efficiency.
- Removal of such words does not show any negative consequences on the model we train for our task.

“I have never tasted a pizza like that before.”



Word Tokenization

["I", "have", "never", "tasted", "a", "pizza", "like", "that", "before"]



Identify stop words

["I", "have", "a", "like", "that", "before"]

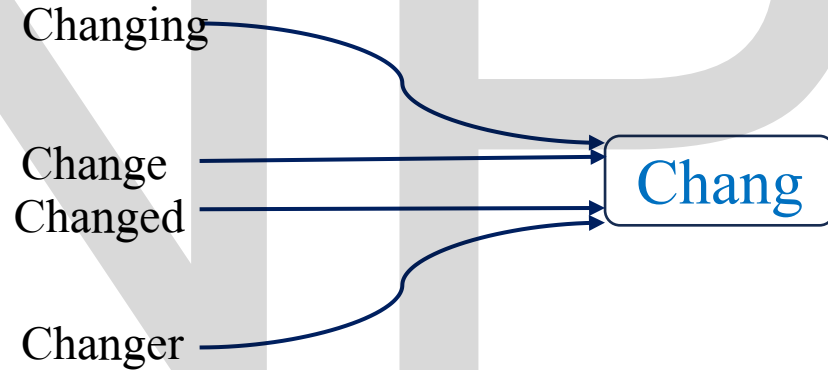


Final Output

“never tasted pizza”

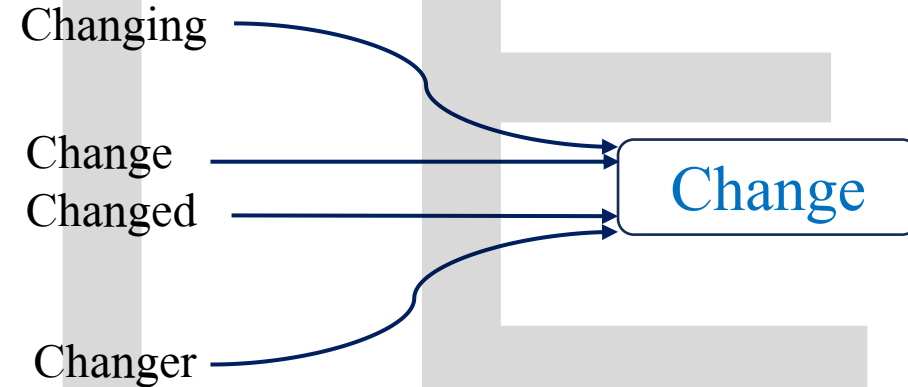
Stemming and Lemmatization

- ✓ Stemming is a technique where words are reduced to their base or root form by removing suffixes or prefixes.
- ✓ Lemmatization is a sophisticated technique that transforms words to their base forms, known as lemmas, considering the word's meaning and context.



Stemming

Doesn't have linguistic knowledge



Lemmatization

Have linguistic knowledge

Parts of Speech (POS) Tagging

- ✓ POS involves assigning a grammatical category (such as noun, verb, adjective, etc.) to each word in a sentence.
- ✓ Helps machines understand the **syntactic structure** of sentences, which is crucial for tasks like:

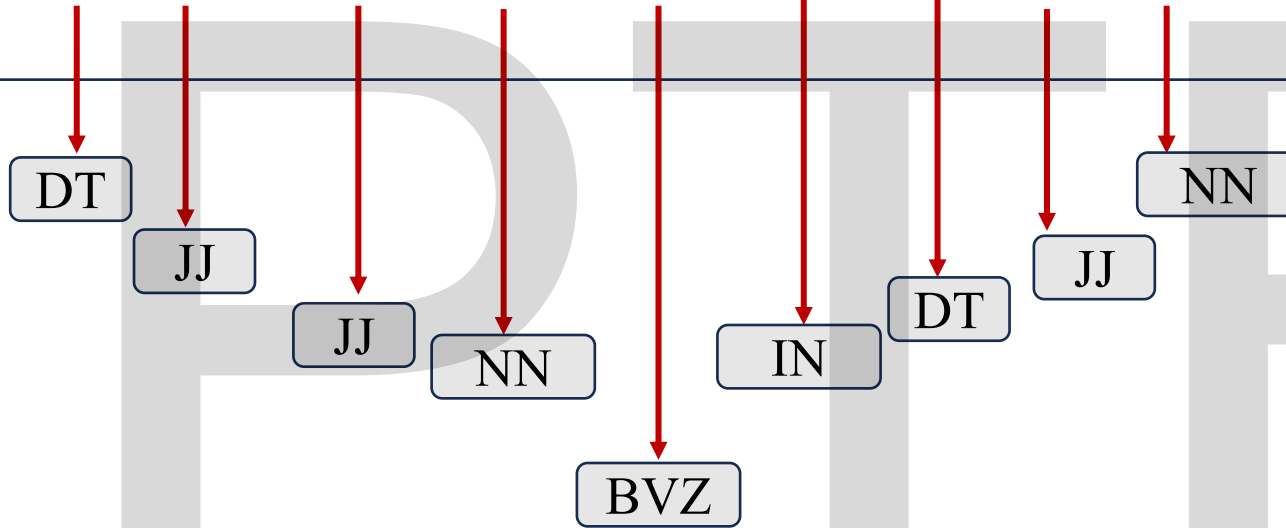


grammar and structure
(the rules for arranging words/symbols in a sentence)

- ✓ Machine Translation
- ✓ Information extraction
- ✓ Question answering
- ✓ Text to speech conversion

Parts of Speech (POS) Tagging

A Quick Brown Fox jumps over a lazy dog



DT → determiner

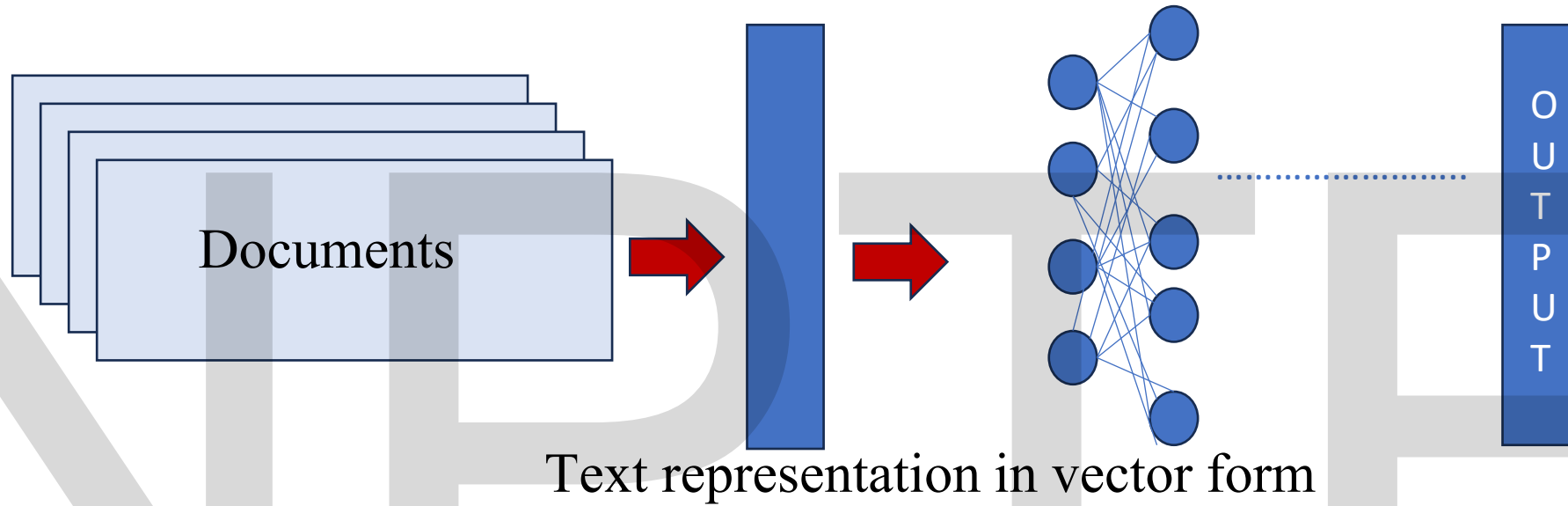
JJ → adjective

NN → noun

BVZ → verb

IN → Preposition

Text Representation in NLP



Text representation involves converting raw text data into machine-readable form.

Discrete text
representation

Distributed text
representation

Discrete Text Representation

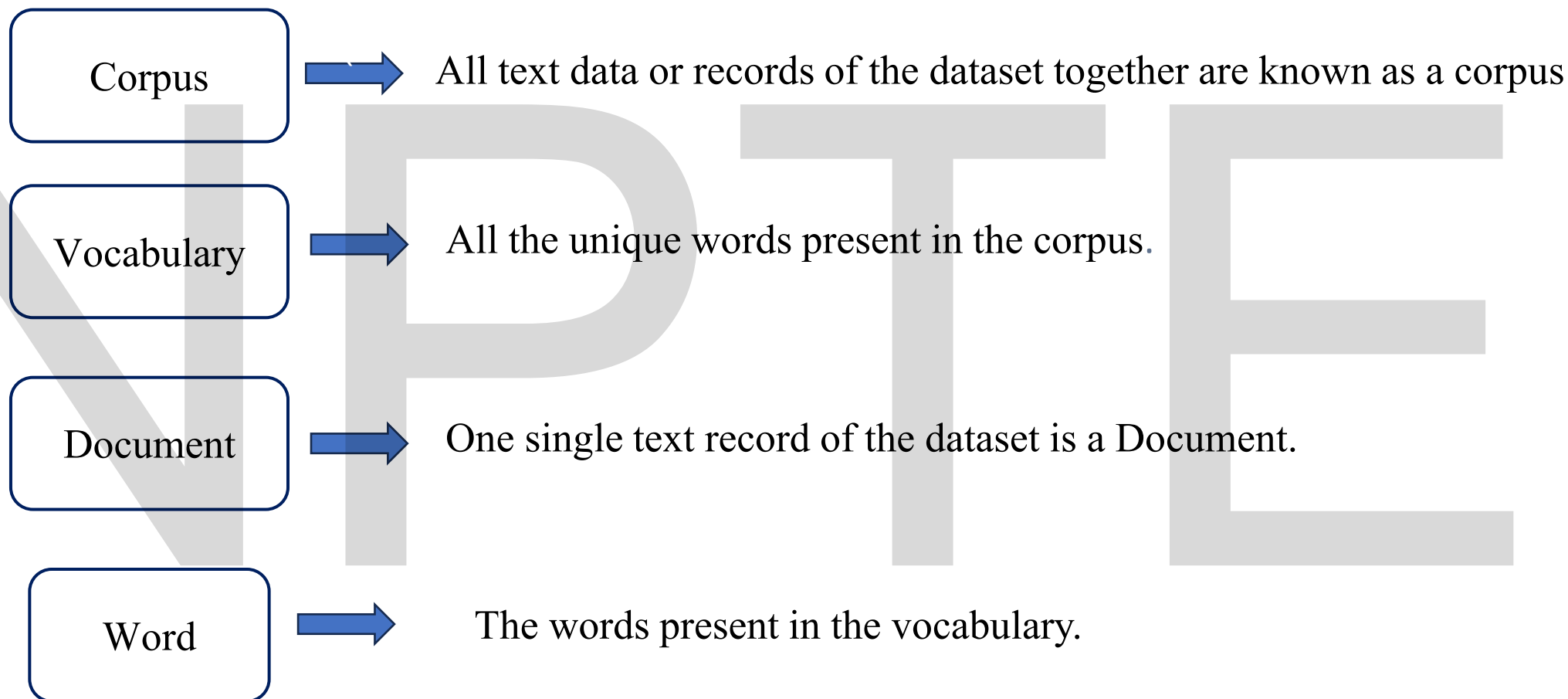
What is Text Representation?

Text representation is a technique used in NLP to transform raw text into numerical features that capture information needed for computational processing and learning.

Technics of DiscreteText Representation

- ✓ One Hot Encoding
- ✓ Bag of words
- ✓ TF-IDF (Term Frequency – Inverse Document Frequency)
- ✓ Ngrams

Key Terminology Related to Text Representation



One-Hot Encoding

One Hot Encoding is a method for converting **categorical variables** into a binary format.

↓
Representing qualities and characteristics

It changes every word to a $|v|$ dimensional vector.

Categorical Variable	Red	Blue	Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1

Now each color is represented as a vector:

$$v_{\text{Red}} = [1, 0, 0]$$

$$v_{\text{Blue}} = [0, 1, 0]$$

$$v_{\text{Green}} = [0, 0, 1]$$

Why One-Hot Encoding is required?

If we used numbers like Red=1, Blue=2, Green=3, the model might think $\text{Green} > \text{Blue} > \text{Red}$, which isn't true. -**Eliminates ordinality**

Limitations of One-Hot Encoding

- ✓ It is a large dimensional sparse vector representation.

Example.

The quick brown fox jumps over a lazy dog.

Vector representation of fox: [0,0,0,1,0,0,0,0,0]

- ✓ Also, it treats each word as independent of other word, even if both the words are together

$$\mathbf{v}_{notebook} = \mathbf{v}_{note} + \mathbf{v}_{book}$$

Bag of Words

The **Bag of Words** model is a way to represent text data numerically so that algorithms can process it.

- Treats each document (sentence, paragraph, etc.) as a “bag” of words.
- Ignores grammar and word order.
- Only cares about the **frequency** (or presence) of words.

Bag of Words(BoW)

Example:

Sentence 1:I love NLP

Sentence 2: I love Deep Learning

Vocabulary

[I, love, NLP, Deep Learning]

	I	Love	NLP	Deep	Learning
Sentence 1	1	1	1	0	0
Sentence 2	1	1	0	1	1

Vectors for Sentence 1 and 2:

Sentence 1 : [1, 1, 1, 0,0]

Sentence 2 : [1, 1, 0, 1,1]

Limitation of Bag of Words

Sentence 1: Cat eats fish

Sentence 2: Fish eats cat

Vocabulary

[Cat, eats, fish]

	Cat	eats	fish
Sentence 1	1	1	1
Sentence 2	1	1	1

Sentence 1 : [1, 1, 1]

Sentence 2 : [1, 1, 1]

} Word order information is lost

Term Frequency-Inverse Document Frequency(TF-IDF)

- ✓ TF-IDF is a statistical measure used to evaluate the importance of a word in a document within a collection or corpus of documents.

It combines two key factors:

- ✓ Term Frequency (TF) and
- ✓ Inverse Document Frequency (IDF).

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t, D) = \log\left(\frac{\text{Total Number of documents in corpus } D}{\text{Number of documents containing term } t}\right)$$

Example of TF-IDF

Document 1: "The cat sat on the mat."

Document 2: "The dog played in the park."

Document 3: "Cats and dogs are great pets. "

Total number of documents in the corpus(D) =3

Calculation of TF score for the word “cat”

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

	No of times cat appears	Total number of terms	TF score
Document 1	1	6	1/6
Document 2	0	6	0/6
Document 3	1	6	1/6

Example of TF-IDF

Calculation of IDF score for the word “cat”

Total number of documents in the corpus (D):	3	IDF(cat, D)
Number of documents containing the term "cat":	2 (Document 1 and Document 3).	$\log \frac{3}{2} \approx 0.176$

TF-IDF(t, d, D) = TF(t, d) * IDF(t, D)

$$\text{IDF}(t, D) = \log\left(\frac{\text{Total Number of documents in corpus } D}{\text{Number of documents containing term } t}\right)$$

For Document 1: TF-IDF (cat, Document 1, D): $0.167 * 0.176 = 0.029$

For Document 2: TF-IDF(cat, Document 2, D): $0 * 0.176 = 0$

For Document 3: TF-IDF (cat, Document 3, D): $0.167 * 0.176 = 0.029$

NPTEL

Summary